

FC 开发指南 (C 语言)

Function Component Development Guideline (C Language)

目录

文档修订记录	4
1 目的	5
2 适用范围	5
3 定义和缩写	5
3.1 缩写	5
3.2 定义	5
4 FC 架构设计	5
4.1 FC 文件结构	5
4.1.1 文件定义	6
4.1.2 文件包含关系	7
4.2 基本 FC 分层设计	9
4.2.1 实现接口层 realize interface layer	9
4.2.2 功能层 function layer	11
4.2.3 依赖接口层 dependency interface layer	11
4.2.3.1 可选的固化代码	11
4.2.3.2 宏替代函数	12
4.2.3.3 标准接口格式函数配置	13
4.2.3.4 callout 函数	13
4.3 复杂 FC 分层设计	15
4.3.1 通讯层 communication layer	16
4.3.2 寄存器抽象层 register abstraction layer	17
4.3.2.1 寄存器数据结构 register data structure	17
4.3.2.2 寄存器通讯帧结构和处理 register communication frame structure and process	17
4.3.3 原子功能层 atomic function layer	18
4.3.4 集成功能层 integrated function layer	19
5 FC 功能要求	19
5.1 初始化	19
5.2 开发错误追踪 (DET)	19
5.3 内部错误标记 (代码逻辑)	19
5.4 内部故障标记 (代码功能)	19
5.5 故障恢复 (可选代码功能)	20
6 FC 代码编写规范	20
6.1 ISO 26262-6 映射关系	21
6.2 代码框架规范	22
6.3 代码格式规范	23
6.4 代码设计规范	25
6.4.1 静态检查规范	26
6.4.2 禁止规则	27
6.4.3 强制规则	28

6.4.4	推荐设计方法	31
7	支持/相关性文件	33
8	附件	33

文档修订记录

[illegible]

1 目的

本文档用于规范 BSW 层级 FC 软件设计和代码编写（包括并不限于外设芯片驱动，复杂驱动等 FC）

2 适用范围

本文档适用于 BSW 层级 FC8 开发工程师作为设计指导和规范参考（guideline）。

3 定义和缩写

3.1 缩写

缩写	描述
<i>FC</i>	<i>Function Component</i>
<i>BSW</i>	<i>Basic Software</i>
<i>ExtDev</i>	<i>External Device</i>
<i>Cdd</i>	<i>Complex Device Driver</i>

3.2 定义

名称	描述

4 FC 架构设计

FC 架构设计要先于 FC 代码编写。FC 架构的作用为规范 FC 对外接口，即将 FC 中和集成相关的部分划分出来并定义好，在后续的 FC 代码编写和升级的过程中，和集成工作解耦（FC 非接口类代码更新不影响集成）。

注意：FC 对外接口的定义，不仅是接口函数，还包括：

1 站在集成角度看，需要 FC 提供的集成的信息。

2 站在 FC 角度看，基于 FC 功能实现，对集成提出的特殊要求。

FC 架构设计包含以下几部分：文件结构定义，接口定义和 memory 段定义。

4.1 FC 文件结构

FC 架构设计阶段需要明确定义 FC 文件结构（文件内容和包含关系）。FC 开发指南提供的 FC 文件结构定义规则，按照功能实现、数据、类型、接口等多个方面划分，并定义各类文件内容和包含关系。以下为 FC 文件结构定义规则详述，规则能够支持覆盖从简单到复杂的 FC 代码开发（特殊情况允许 overrule）。

4.1.1 文件定义

各部分文件定义规则如下:

类型	必需性	文件命名	包含 memory 段	说明
Code	必需	FC[_Desc].c FC[_Desc].h	RAM 数据段 ROM 数据段 CODE 段	①
Cfg	必需	FC[_Desc]_Cfg.c FC[_Desc]_Cfg.h FC[_Desc]_CfgData.h	ROM 数据段 (配置参数)	②
Cal i	可选	FC[_Desc]_Cal i.c	ROM 数据段 (标定参数)	③
Type	必需	FC[_Desc]_Types.h	无	④
Callout	可选	FC[_Desc]_Callout.c FC[_Desc]_Callout.h	CODE 段	⑤
Mem	必需	FC_MemMap.h	定义所有 FC memory 段	⑥

注意: 根据元语言规则, []内表示可选字段。

① FC 代码文件, 作为实现 FC 功能的主体, 内容包含内部变量定义 (static variable), FC 实现的接口函数定义 (realize interface), 实现 FC 功能。每个 FC 至少包含一组 FC.c/.h, 复杂的 FC 可增加多组代码文件, 命名方式为 FC_Desc.c/.h (一组 FC 代码文件允许包含多个.c/.h, 不强制要求成对定义), Desc 为功能描述字段 (使用驼峰命名法), 当定义多组代码文件时, FC 实现的接口函数定义必需放在 FC.c/.h 中。

注意: BSW FC 不要定义 global variable 用作变量接口。

② FC 配置参数文件, 内容包括配置参数的定义 (const variable) 在 FC_Cfg.c 中, 配置参数和标定参数 (如果有) 的声明在 FC_CfgData.h 中, 配置宏定义 (宏开关或宏值) 在 FC_Cfg.h 中。每个 FC 至少包含一组 FC_Cfg.c/.h 和 FC_CfgData.h, 复杂的 FC 可增加多组配置参数文件, 命名方式为 FC_Desc_Cfg.c/.h 和 FC_Desc_CfgData.h, Desc 为功能描述字段 (使用驼峰命名法)。

注意: 根据复杂 FC 分层设计概念, 每一层实现 FC 功能的主体是 FC 代码文件, 可以为此配备一组 FC 配置参数文件 (如果需要)。

③ FC 标定参数文件, 内容包括标定参数的定义 (const variable) 在 FC_Cali.c 中。根据功能需求, 每个 FC 可包含一个 FC_Cali.c 或多个 FC_Desc_Cali.c 标定参数文件。

注意: 一般情况下 BSW FC 不需要标定参数定义。如果需要定义多组 FC 标定参数文件, 注意事项同 FC 配置参数文件描述 (对于 BSW 层级的 FC 来说, 标定变量也是一种配置参数, 标定变量的定义, 与配置参数文件中的变量定义没有区别)。

④ FC 类型文件, 内容包括 FC 通用的宏定义, 特殊值域的基础类型声明, 结构体声明, 枚举声明等。每个 FC 至少包含一个 FC_Types.h, 复杂的 FC 可增加多个类型文件, 命名方式为 FC_Desc_Types.h, Desc 为功能描述字段 (使用驼峰命名法)。

注意: 如果需要定义多组 FC 类型文件, 注意事项参考 FC 配置参数文件描述。当定义了多个 FC 类型文件是, 必需有一个 FC_Gen_Types.h 文件包含共用的 (general) 类型定义 (例如: 空指针宏定义, 核 ID 宏定义等, 这些宏定义原本在简单 FC 的 FC_Types.h 中, 由于复杂 FC 的有多组文件需要引用这些共用的类型定义, 因此从 FC_Types.h 转移到 FC_Gen_Types.h)。

⑤ FC callout 文件, 内容包括 FC 依赖功能函数定义和声明 (dependency interface)。根据功能需求, 每个 FC 可包含一组 FC_Callout.c/.h 或多个 FC_Desc_Callout.c/.h。

注意: 一般情况下 BSW FC callout 文件仅需要实现 FC dependency 功能 (同步调用), 计算过程变量使用函数内局部变量, 不需要定义内部变量 (static variable)。根据复杂 FC 分层设计概念, FC callout 代码仅属于复杂 FC 分层设计的 dependency 层, 一般不会需要定义多组 callout 文件。

⑥ FC memory 文件, 内容包括 FC 其他文件定义的段宏名的编译器重定义, 或集成重定义 (关于编译器的 #pragma 相关操作也可放在 FC_MemMap.h, 如特定代码段的特殊编译器优化)。每个 FC 有且仅有一个 FC memory 文件 FC_MemMap.h。

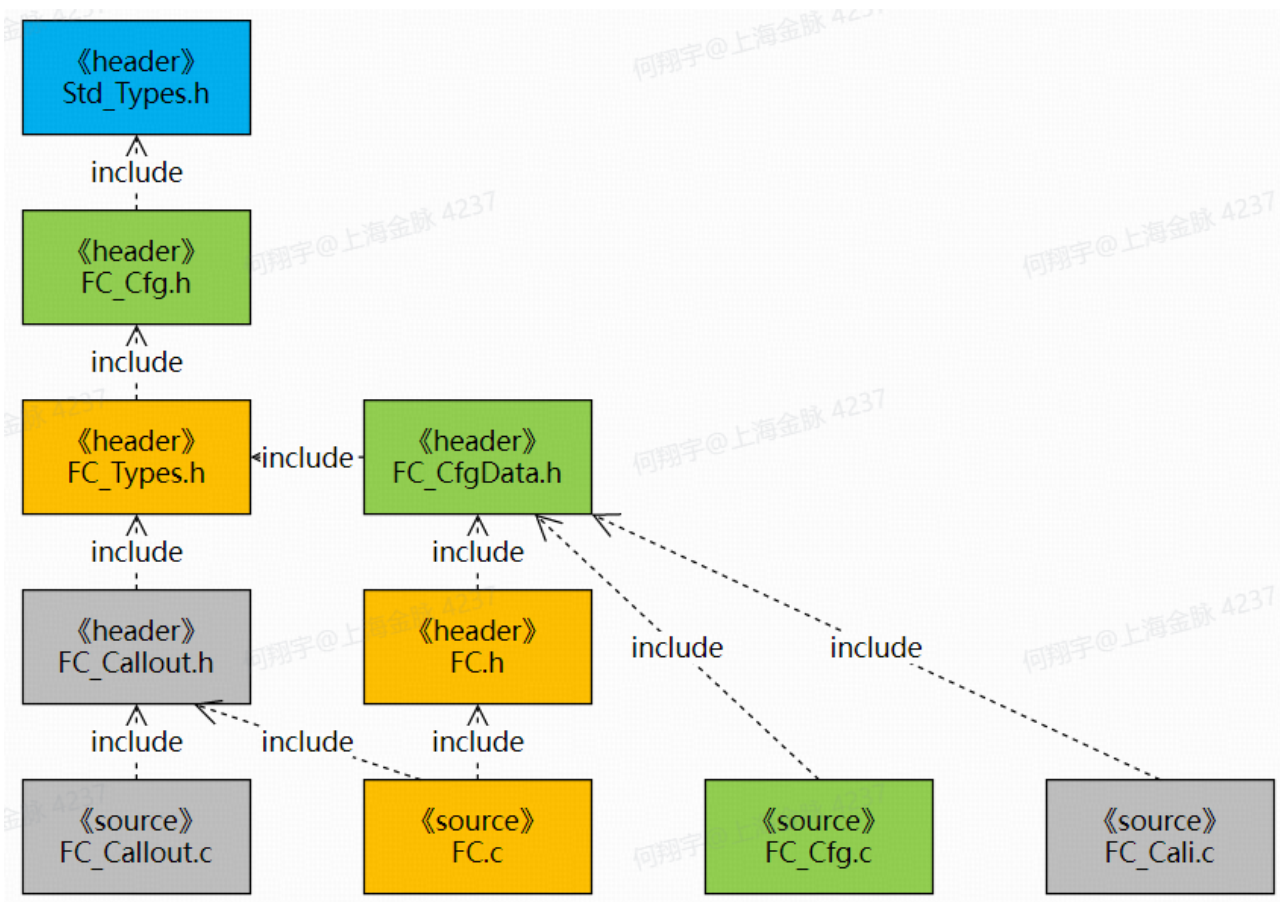
4.1.2 文件包含关系

基本 FC 文件包含关系如下:

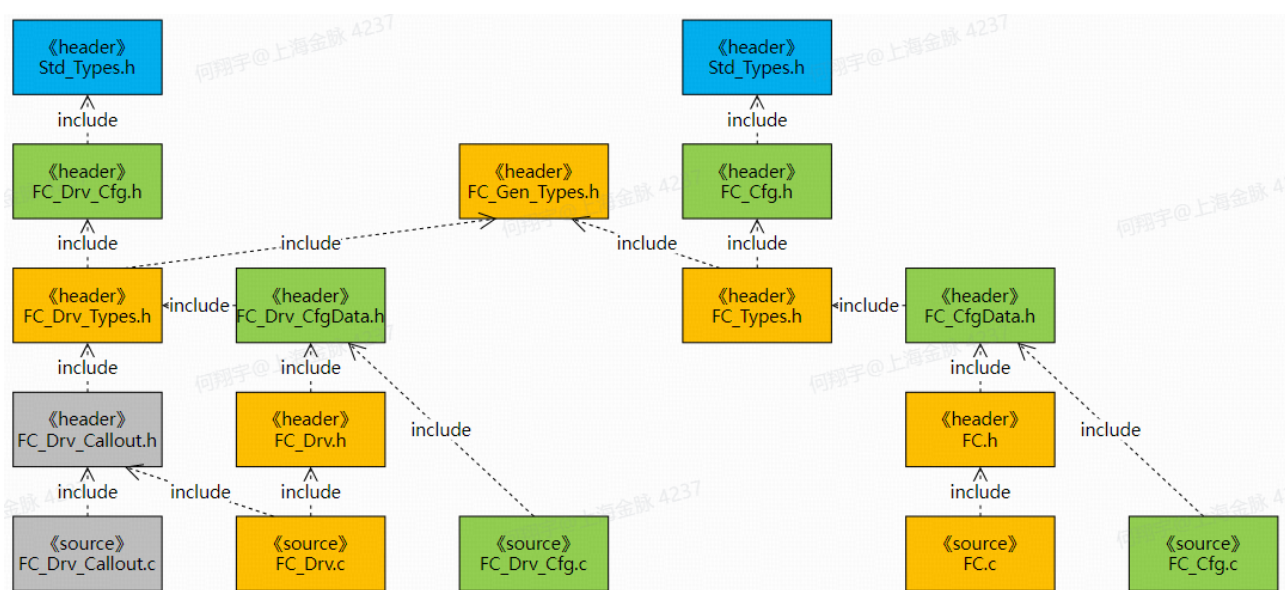
注意: 下图文件按颜色分类, 橙色和绿色是 FC 必需文件, 橙色是静态文件, 绿色是配置文件。灰色是可选文件 (可选文件也属于配置文件)。蓝色是每个 FC 必须包含的标准头文件 (Std_Types.h), FC_MemMap.h 文件默认在各个 FC 文件中定义 memory 段宏名后被包含, 图

中不再累述。

注意：以上颜色分类仅用于本文档图示区分不同文件，不代表 FC 架构文件作图颜色。



根据复杂 FC 分层设计概念，示例文件包含关系示例如下（不含 FC 标定参数文件）：



从以上 FC 文件定义和 FC 文件包含关系可以看出，复杂 FC 分层设计的文件结构可以看做是基本 FC 的组合，每一层的核心是 FC 代码文件，其他类型的文件根据需求配备，例如某些

功能实现代码无需配置参数, 或仅使用基础类型, 无需定义 FC 类型文件, 因此文件包含关系可以根据实际情况调整。

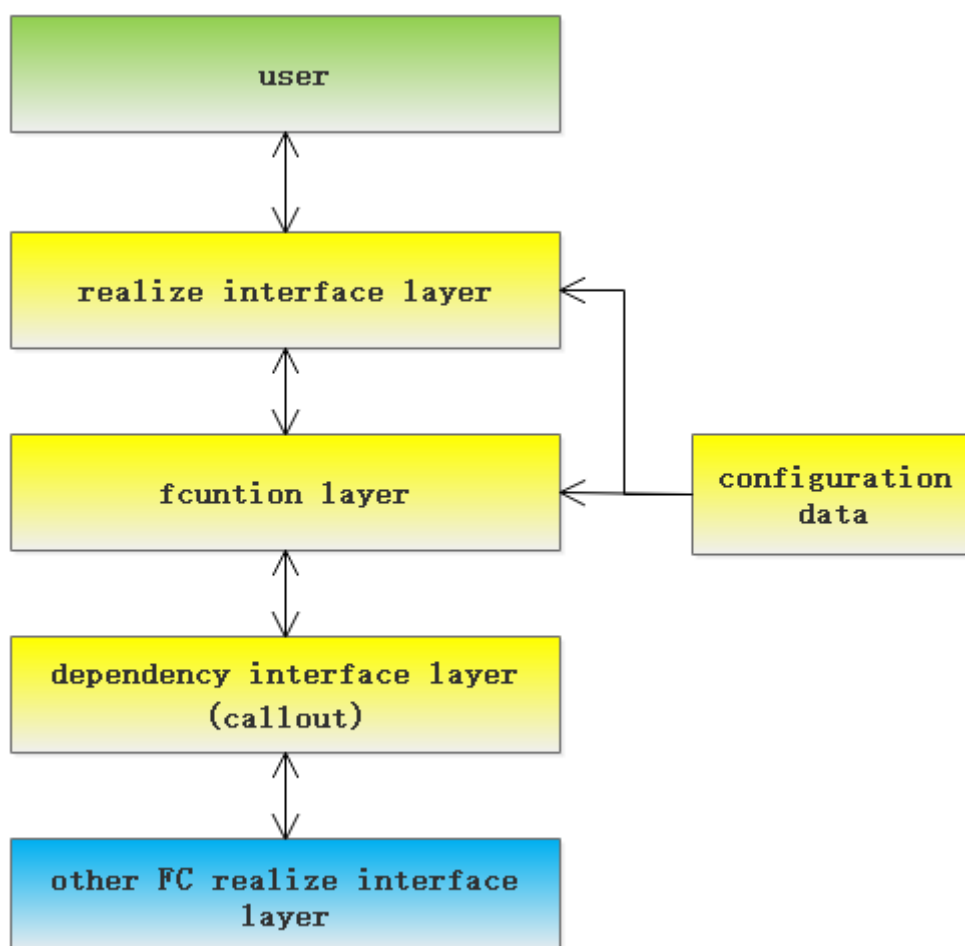
推荐设计方法如下:

(1) FC callout 文件只会在 dependency 层需要。

(2) 层与层之间只使用函数做接口, 不使用变量做接口, 方便复杂 FC 层级功能解耦和分层测试。

4.2 基本 FC 分层设计

具有一定复杂性的 FC, 内部代码需要按层次化设计, 将代码分为多个层次, 每个层次根据定义完成不同的功能。一般来说 FC 内部层级拆分如下:



4.2.1 实现接口层 realize interface layer

实现接口层, 代码主要包括 FC 实现的接口函数, 供 user 调用, 包括并不限于 init 函数, mainfunction 函数, 获取输入信号或设置输出信号接口函数。除了 init 函数和 mainfunction

函数外，其他接口函数一般为异步函数，异步函数功能如下：

(1) 将 user 提供的设置输出信号值保存到内部全局变量中，由 mainfunction 来处理并完成内部动作和功能。

(2) 将 user 需要获取的信号值填入 user 提供的获取输入信号地址中（指针）。user 需要读取的信号值由 mainfunction 函数在上一个周期运行后得到并保存到内部全局变量中（如果 mainfunction 未运行，user 将得到此信号的初始化值）。

注意：功能简单或者实时性能简单或者实时性要求高的场景，会使用同步接口函数。同步接口函数的作用为，当 user 调用此接口函数时，函数将贯穿整个 FC 层级，解析接口输入信息，完成内部逻辑和功能处理，并调用已配置好的依赖接口层函数完成功能。

实现接口层的函数除 init 函数外，其他函数需要考虑防御性策略，用于防止和排查集成造成的错误，例如：

(1) 是否已完成初始化检查，防止 init 函数未被调用时，直接调用其他函数导致功能错误。

(2) 输入形参是否在范围内，防止超范围值导致功能错误。

(3) 输出形参（指针）是否为空，防止写空指针造成软件 TRAP。

当检查到以上错误时，函数不运行功能，保存错误信息到内部全局变量（DET 变量），并通过返回值报 E_NOT_OK（除 mainfunction 外，其他接口函数需要定义标准返回值类型 Std_ReturnType）。

注意：此处的防御性策略是 FC 对外表现出防御性，只在实现接口层考虑。FC 调用内部函数时不需要检查，调用内部函数形参输入的正确性由内部设计和测试保证。另外实现层接口的检查代码可以通过 DET 宏开关控制预编译，选择启用或屏蔽此部分代码，当 DET 关闭时，实现接口层函数不做防御性检查，并始终返回 E_OK。

经验提示：

(1) 以多核多实例的 FC 实现接口层异步函数为例，代码主要完成的功能为，通过 user 输入的 ID，找到此 ID 的配置信息，找到实际需要操作的某个核的某个实例的某个信号的内部变量，将内部变量值写入获取输入信号地址（输出形参），或者从设置输出信号读取值写入内部变量。至此实现接口层函数工作结束，内部处理交给 mainfunction 和其他 static 函数完成。

(2) 实现接口层防御性策略的返回值 (Std_ReturnType) 一般来说用于表示防御性检查的结果, 和 FC 功能, FC 提供的信号无关。硬件故障导致无法运行或者读取信号判定为无效等情况下, 需要通过其他途径记录和上报, 不需要通过接口函数的返回值上报。

4.2.2 功能层 function layer

功能层, 代码实现 FC 功能, 主要为内部函数 (static), 功能简单的 FC 的 mainfunction 函数也可以算作功能层代码。

功能层代码需要考虑复用性, 将功能拆分到较小的可复用的原子化函数, 可提升代码复用性和降低圈复杂度。

较复杂的外设驱动 FC, 功能层还可以细分, 在后续章节说明。

注意: 功能层函数不需要考虑防御性策略 (mainfunction 除外)。

4.2.3 依赖接口层 dependency interface layer

大部分 FC 在实现功能时, 都需要依赖 (调用) 其他 FC 的接口函数。此部分代码作为配置代码实现, 需要在灵活性, 易配置, 代码效率等方面平衡决策。以下例举了几种依赖 (调用) 接口实现方式。

4.2.3.1 可选的固化代码

当被依赖 (调用) 的其他 FC 相对固定时, 且变化在一定范围内 (例如: 几款主流 microcontroller 的 MCAL 接口函数), 可以将被调用接口函数、类型定义、处理代码固定到当前 FC 中, 并通过预编译宏进行选择。

比如依赖 (调用) MCAL 接口时, 可固化调用 MCAL 接口的代码, 并通过预编译宏来配置选择不同的 MCAL 接口 (例如: AURIX 和 AURIX 2G 某些模块接口有差异)

```
#if (GP_IOMCUPWM_SPEC_MCUIF == GP_IOMCUPWM_SPEC_MCUIF_IFX2GMCAL)
    uint32 PeriodTick_u32 = 0U;
    uint32 DutyTick_u32 = 0U;
#elif (GP_IOMCUPWM_SPEC_MCUIF == GP_IOMCUPWM_SPEC_MCUIF_UNDEFINED)
#endif
```

```
#if (GP_IOMCUPWM_SPEC_MCUIF == GP_IOMCUPWM_SPEC_MCUIF_IFX2GMCAL)

    /*convert time(us) to tick*/
    PeriodTick_u32 = EcuPed_u32 * GP_IOMCUPWM_US_TO_NS;
    PeriodTick_u32 = PeriodTick_u32 / GP_IOMCUPWM_NS_PER_TICK;
    DutyTick_u32 = EcuDuty_u32 * GP_IOMCUPWM_US_TO_NS;
    DutyTick_u32 = DutyTick_u32 / GP_IOMCUPWM_NS_PER_TICK;
    Pwm_17_GtmCcu6_SetPeriodAndDuty\
    ((Pwm_17_GtmCcu6_ChannelType)Gp_IoMcuPwm_cfgContainer_votst.PwmChlCfg_pul6[Id_u16],
     (Pwm_17_GtmCcu6_PeriodType) PeriodTick_u32,
     DutyTick_u32);

#elif (GP_IOMCUPWM_SPEC_MCUIF == GP_IOMCUPWM_SPEC_MCUIF_UNDEFINED)

#endif
```

此方法灵活度低，配置简单，代码效率高，被依赖（调用）接口可选范围增加时，需要更新源码。

注意：为减少集成编译问题，FC 使用此方法时，需要提供 UNDEFINED 选择配置，保证代码工程在不提供被依赖（调用）的 FC 时，也能编译通过。

4.2.3.2 宏替代函数

当被依赖（调用）的函数接口简单时（无形参，无返回值，无特殊数据类型），可以在 FC 源码中通过宏来实现，在 Cfg.h 中配置宏为被依赖（调用）函数。

比如进入退出临界区宏，配置为开关中断函数，或 0s 挂起恢复中断函数。

```
/*critical*/
#define GP_DEMO_ENTER_CRITICAL()      Os_SuspendAllInterrupts()
#define GP_DEMO_EXIT_CRITICAL()       Os_ResumeAllInterrupts()
```

注意：形参简单的函数也可以用功能宏代替（不使用带参数宏）。此方法使用的宏在配置为空时，也能编译通过。

```
#define GP_DEMO_SET_ENABLE_PIN        Dio_WriteChannel(DO_DEMO_EN, STD_HIGH)
```

此方法灵活度高，配置简单，代码效率高，仅适用简单的被依赖（调用）接口。此方式由于无形参传入，因此不能用于可配置数量的多实例的控制，以上图为例，如果要分别控制多个 enable pin，宏替代函数无法实现多实例化代码。宏替代函数一般用于统一的操作控制，并且被调用函数本身也支持多实例调用（进入退出临界区代码就是很适合的应用实例）。

4.2.3.3 标准接口格式函数配置

当被依赖（调用）的函数接口是软件平台定义的标准信号接口时，可以在 `Fc_Demo_Cfg.c` 中配置被依赖（调用）的函数名字（同类信号接口函数的形参和返回值都一致，只有函数名称根据 FC 名定义会有不同）。

此方法灵活度低，配置简单，代码效率高，仅适用软件平台接口。

注意: FC 使用此方法时，需要提供 NULL 指针或 NULL 函数定义，保证代码工程在不提供被依赖（调用）的 FC 时，通过填充 NULL 指针或 NULL 函数，也能编译通过。

4.2.3.4 callout 函数

当被依赖（调用）的函数接口定义不明确时，可以在 FC 源码中调用本 FC 的 callout 函数，callout 函数形参数量和类型根据本 FC 需求来定，callout 函数内容在软件集成时作为配置代码编写。

此方法灵活度高，配置复杂，代码效率低（可能存在参数转换运算），适用范围广。

callout 部分属于配置代码（`FC_Callout.c` 文件）。大部分 FC 实现功能需要调用其他 FC 的接口函数（控制输出，获取输入等）。

callout 函数的设计方法，需要从 FC 本身功能出发，提出功能需求，不需要基于现有的其他 FC 提供的接口函数去设计。

例如，FC 需要同步发送 SPI 数据，callout 函数只需要提供收发数据的缓存地址和数据长度（callout 函数形参），不需要关注 AUTOSAR 的 sequence/job/channel 等概念。FC 在集成配置时，如果工程使用了 MCAL，在 callout 函数中添加配置代码，直接调用 MCAL 的 SPI 同步发送函数，并填入相应的 sequence 号，此 sequence 号不需要在 FC 配置数据中体现。

同样，FC 需要控制芯片的某个管脚电平时，callout 函数只需要提供期望的输出电平值（callout 函数形参），不需要关注实现过程，FC 在集成配置时，如果工程使用了 MCAL，在 callout 函数中添加配置代码，直接调用 MCAL 的 DIO 写通道函数，并填入相应的 channel 号和电平值，此 channel 号不需要在 FC 配置数据中体现。

注意: 如果控制 IO 口线路在 ECU 板上有翻转电路，翻转电平的逻辑应该写在 callout 函数中。因为，站在 FC 提出功能需求的角度看，FC 希望控制芯片的管脚为高电平，callout 函数输入的形参就是高电平，由于 ECU 板上有翻转电路，实际需要调用 MCAL DIO 接口输出低电

平, 这个判断并翻转输出的代码需要在 *callout* 函数内完成。

callout 函数作为 FC 实现功能的一种途径, 需要注意调用其他 FC 接口函数可能存在不能实现功能的情况 (例如: 调用传入参数错误, 硬件故障等), 因此 *callout* 函数需要检查被调用的其他 FC 接口函数的返回值, 确认功能是否实现 (如果其他接口函数带此类返回值)。 *callout* 函数将检查结果反馈给调用此 *callout* 函数的 FC 内部函数, FC 需要根据结果做相应处理。

注意: callout 返回结果可以简单定义 boolean 变量, TRUE/FALSE 表示执行功能已完成/未完成, 如果需要根据多种反馈状态做处理, 可以自定义数值变量和多个状态值。当 callout 函数调用了某些不带返回值的函数, 无法判断运行结果时, callout 函数可以固定返回功能已完成 (例如: 调用 DIO write channel 函数)。

注意: 支持多核运行的 FC 代码中, 获取 core ID 的 callout 函数不需要带判断运行结果的返回值。由于多核 FC 的 core ID 作为内部变量数值索引, 如果 core ID 不正确, FC 后续代码无意义并且无法记录错误变量, 因此要求获取 core ID 函数保证正确性。

经验提示:

(1) 某个项目中, FC 需要控制外部传感器的 EN 管脚电平, 接插件上没有 DIO 通道, 项目中使用预留 PWM 输出通道代替, FC 的 *callout* 函数输入期望电平值, 将在函数内部转换为调用 MCAL 的 PWM 接口, 需要高电平时 PWM 输出占空比 100%, 需要低电平时 PWM 输出占空比 0%。

(2) 以多核多实例的 FC 为例, 可能存在某个功能的 *callout* 函数需要覆盖到全部核, 全部实例, 全部信号。此 *callout* 函数需要提供核 ID, 实例 ID, 信号 ID (callout 函数形参)。例如某个多核多实例的 FC 需要实现 PWM 发送功能, 此功能通过 *callout* 实现, *callout* 函数需要提供 core ID, chip ID (实例 ID), channel ID (信号 ID), PWM period, PWM duty (callout 形参)。 *callout* 函数内部会根据 3 个 ID 信息, 找到需要控制的那个 channel, 然后调用 MCAL PWM 接口, 填入对应的 channel 号和周期占空比值。

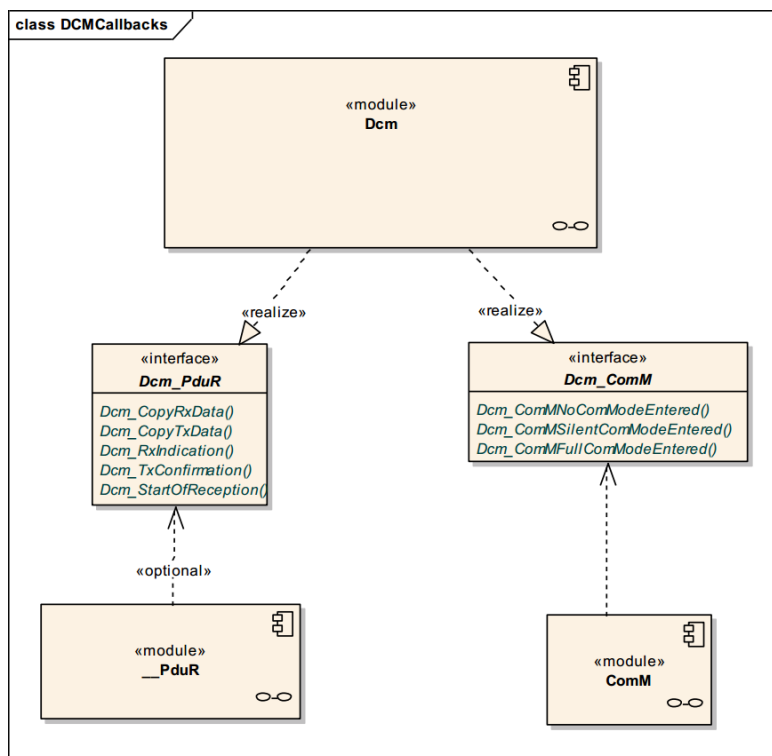
注意: callout 函数和 callback 函数定义区别如下:

参考 AUTOSAR DCM 模块的设计文档《*Specification of Diagnostic Communication Manager*》AUTOSAR Release 4.2.2。文档 8.4 和 8.5 章节对 *callback* 和 *callout* 函数解释, *callback* 函数在本 FC 定义并实现代码功能, 一般交给下一层模块调用 (在某个特殊时刻, 类似通知函

数), *callout* 函数由本 *FC* 提出功能需求 (功能需求影响 *callout* 函数形参设计), *callout* 函数的代码, 在集成环节调用其他 *FC* 提供的接口函数实现。AUTOSAR 截图参考图如下:

8.4 Callback Notifications

This section defines the functions provided for lower layer BSW modules. The function prototypes of the callback functions will be provided in the file Dcm_Cbk.h



8.5 Callout Definitions

Callouts are pieces of code that have to be added to the DCM during ECU integration. The content of most callouts is hand-written code, for some callouts the DCM configuration tool shall generate a default implementation that is manually edited by the integrator. Conceptually, these callouts belong to the ECU Firmware.

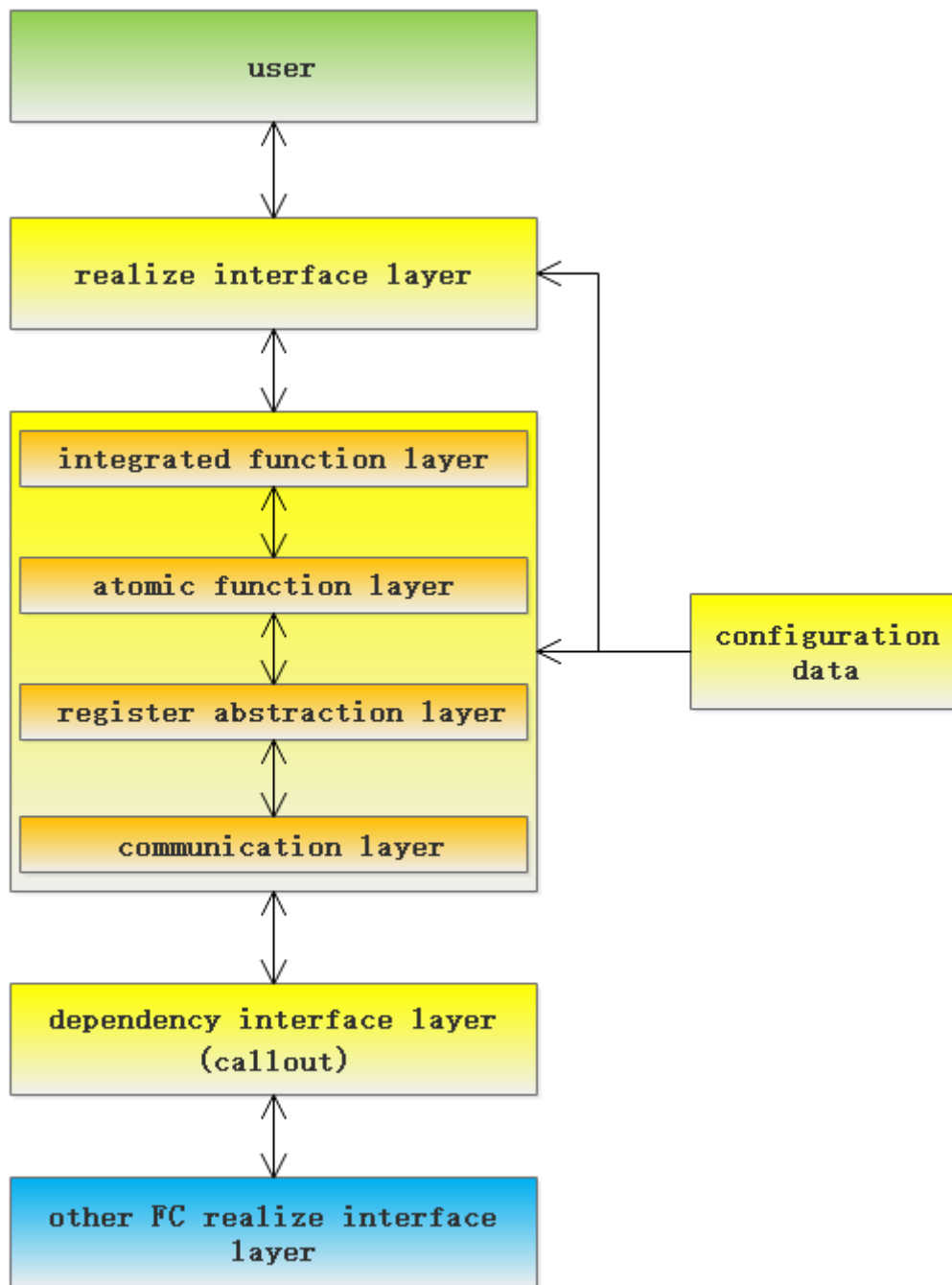
4.3 复杂 FC 分层设计

带有通讯接口 (SPI, UART 等) 和寄存器的外设芯片, 一般需要复杂分层设计。实现接口层和依赖接口层参考基本分层设计章节说明。功能层将被拆分为更细化的层次, 每个层次有明确的定义和分工 (鲁棒性分工), 在实际开发中根据需求决定是否拆分出此层。

根据层次拆分的结果, 可能是拆分出函数 (较简单的层), 可能拆分出文件 (较复杂的层)。当拆分出文件时, 为保证分层代码的独立性, 本层变量定义在本层文件中, 仅供本层使用,

层与层之间使用函数做为接口，不要通过全局变量传递数据，数据类型可以统一定义在 FC_Types.h 中或者本层的头文件中。同样，配置数据也应该分层定义，配置数据较多时，可以拆分出多个配置文件。

FC 内部层级拆分如下：



4.3.1 通讯层 communication layer

通讯层，代码功能包括调用依赖接口层的接口函数，完成数据收发功能。

此层代码需要根据依赖层接口函数反馈结果判断通讯是否完成（如果其他 FC 提供的接口有反馈），同时负责通讯功能的鲁棒性，当依赖层接口反馈通讯出错时，尝试再次通讯，记录事件，尝试失败，记录故障。

如果需要实现异步通讯功能时，此层代码需要提供通讯 mainfunction 供周期调用。

如果通讯数据量较大时，此层代码需要实现通讯数据 FIFO 缓存功能（例如 AFE 的菊花链通讯），将上一层需要收发数据缓存到 FIFO 中，由通讯 mainfunction 逐个完成收发。

4.3.2 寄存器抽象层 register abstraction layer

寄存器抽象层，此层代码会将外设芯片的寄存器抽象为全局变量供功能层使用，寄存器变量定义在此层文件中。寄存器读写通讯帧格式 pack 和 unpack 处理在此层实现，包括通讯帧格式校验，寄存器值校验和鲁棒性功能。

4.3.2.1 寄存器数据结构 register data structure

需要实现单寄存器读写方式（single write, single read），对于一个寄存器，变量定义包括读值变量，读操作状态，写值变量，写操作状态。通过数组管理所有变量。

注意：读寄存器数量和写寄存器数量不一样，需要根据手册定义寄存器读写属性决定（只读，只写，读写）。

根据应用场景，可以增加组寄存器读写方式（group write, group read），对于一组寄存器，变量定义包括读值变量组，读操作状态，写值变量组，写操作状态。

注意：可根据需求定义多个组寄存器读写方式，组寄存器可以和单寄存器重叠，变量不要共用，组寄存器读写操作状态变量为单个变量，不是数组。

4.3.2.2 寄存器通讯帧结构和处理 register communication frame structure and process

需要实现单寄存器读写帧结构，对于一个寄存器，定义单寄存器发送和接收帧格式，每帧固定格式数据在初始化时写入发送缓存（寄存器地址，读写命令等），非固定数据在发送时写入发送缓存（写寄存器值，CRC 校验值等）。

单寄存器写功能操作顺序（以同步 SPI 为例）：

- （1）从写值变量读取值，组合到发送帧缓存，写操作状态修改为“进行中”。
- （2）计算发送帧校验（如果通讯帧带有校验）。

(3) 调用 `dependency layer` 接口发送数据。

(4) 检查 `dependency layer` 接口反馈状态, 反馈发送故障, 停止发送并记录故障, 反馈发送完成, 进入下一步。

(5) 检查写操作 `reply` 数据有效性 (如果通讯帧带有校验), 根据校验结果决定后续操作 (鲁棒性)。

(6) 校验错误, 尝试重写, 重写错误报故障 (重写次数可配置)。

(7) 校验正确或尝试重写校验正确, 尝试回读检查 (回读值和预期值比较, 预期值可配置), 回读检查错误报故障

注意: 写寄存器回读检查功能需要支持可配置, 例如 `write only` 寄存器不需要做回读检查。

单寄存器读功能操作顺序 (以同步 SPI 为例):

(1) 清除发送帧缓存中的读值, 读操作状态修改为“进行中”。

(2) 计算发送帧校验 (如果通讯帧带有校验)。

(3) 调用 `dependency layer` 接口发送数据。

(4) 检查 `dependency layer` 接口反馈状态, 反馈发送故障, 停止发送并记录故障, 反馈发送完成, 进入下一步。

(5) 检查读操作 `replay` 数据有效性 (如果通讯帧带有校验), 根据校验结果决定后续操作 (鲁棒性)。

(6) 校验错误, 尝试重读, 重读错误报故障 (重读次数可配置)。

(7) 校验正确, 从 `reply` 缓存中提取读数据到读变量值, 读操作状态修改为“读完成”。

4.3.3 原子功能层 `atomic function layer`

原子功能层, 此层代码需要调用寄存器抽象层接口, 完成原子功能 (初始化, 读状态, 写控制输出等)。同时负责实现原子功能鲁棒性。

原子功能校验和寄存器抽象层校验不同, 寄存器抽象层负责保证寄存器读写正确, 原子功能校验需要保证功能正确 (检查寄存器值符合功能预期)。校验结果不符合时, 可尝试重操作功能。

4.3.4 集成功能层 integrated function layer

集成功能层，此层代码需要组合和调用原子功能，完成更复杂的功能操作序列。根据应用场景需求，集成功能层需要支持原子功能组合和操作序列的可配置，需要支持调度时间可配置。集成功能层一般不需要考虑鲁棒性，但是需要记录操作序列过程非预期错误。

5 FC 功能要求

5.1 初始化

FC 需要有初始化功能，最少包含 FC 内部全局变量初始值赋值。

5.2 开发错误追踪 (DET)

FC 需要有 DET 代码和 DET 宏开关，DET 代码通常为输入形参检查和报错。

5.3 内部错误标记 (代码逻辑)

FC 需要定义内部全局变量，用于记录 FC 内部错误路径。内部错误路径定义为集成或配置造成非预期的输入或操作，导致 FC 正常功能无法实现。举例代码逻辑层面包含 (不限于) 以下情况：

接口形参检查无效 (软件集成问题)。

调用其他 FC 接口返回报错。

非预期的状态机状态值 (全局变量被篡改)。

FC 配置数据错误 (指针为空)。

接口函数被调用时检查到初始化函数未被调用过 (软件集成问题)。

要求各个错误标记值之间不能互相覆盖，变量只在初始化清零，运行过程中检查到错误时置位，此变量可作为 DEM 存储环境变量 (冻结帧数据)。

注意：此部分代码可以通过 DET 开关选择屏蔽。

5.4 内部故障标记 (代码功能)

FC (主要适用于 ExtDev) 需要定义内部全局变量，用于记录 FC 内部故障路径 (可以和内部错误标记为同一个变量)。内部故障路径定义为硬件原因影响的 FC 正常功能无法实现 (硬件故障)。举例代码功能层面包含 (不限于) 以下情况：

调用驱动接口返回报错（无法驱动 MCU 资源，比如 Spi, Uart 等）。

非预期的硬件状态（读取到外设芯片未定义的状态值）。

非预期的硬件状态（无法控制外设芯片进入到预期状态）。

读取到外设芯片报故障（硬件故障）。

要求各个错误标记值之间不能互相覆盖，变量只在初始化清零，运行过程中检查到错误时置位，此变量可作为 DEM 存储环境变量（冻结帧数据）。

注意：此部分代码不属于 DET 开关选择范围。

5.5 故障恢复（可选代码功能）

为增加软件健壮性，FC（主要适用于 ExtDev）可以在代码内部增加故障恢复功能（故障恢复操作对 FC 接口无影响），区别于内部故障标记功能（一次性记录），软件可以尝试多次恢复硬件，并清除故障变量（非内部故障标记）。

软件可设定故障次数阈值，可设定尝试次数阈值，故障计数策略（可配置 step 递增）和恢复计数策略（可配置 step 递减或清零）可根据情况制定。

内部故障标记中涉及到的硬件故障均可做尝试恢复。

故障发生并在故障阈值内恢复后，不影响 FC 接口功能，对外界来说无感。FC 需要有变量记录故障发生和恢复的历史数据，此变量可作为 DEM 存储环境变量。

6 FC 代码编写规范

FC 软件设计需要考虑以下通用准则。

Correctness 正确性。

所有 FC 代码必须按照 FC 需求规范来实现，保证实现功能，正确满足需求规范，并且不能添加非需求规范之外的代码。例如，不要把测试代码保留在 FC 代码内。

Maintainability 维护性。

为了提高维护性，需要避免使用容易出现错误的代码（C 语言的复杂指令和用法）。避免难以理解的代码（混乱的逻辑和嵌套关系）。FC 内部使用分层设计，或拆分函数设计，来降低 FC 内部代码的耦合度。

Modularization 模块化。

为了降低模块之间的耦合性，FC 代码需要按照模块化设计。例如，尽量减少模块间有关

联的交互接口，推荐使用单向的，不被其他接口影响的，输入或输出接口。

Readability 可读性。

为了保证良好的可读性，FC 代码内的各个元素（宏，类型，变量，函数等）命名需要按照《G-C046 软件接口命名规范》实施。读者可以通过命名的信息快速理解元素的含义和范围。代码需要足够的注释来帮助读者理解，注释风格需要按照代码编写规范执行。

Real-Time Ability 实时性。

FC 软件设计需要考虑实时性，避免在软件中使用过长的同步延时时间和临界区运行时间。

Integrity 完整性。

FC 软件设计需要考虑在实时处理的情况下保证数据一致性（针对有一致性要求的变量读写），当用于读写变量的 FC 接口函数被多个不同优先级抢占式任务调用时（单核），或者被不同核调用时。

Compiler independence 编译器独立性。

FC 软件设计需要保证代码对编译器的独立性，不依赖于特定的编译器，如果是为了兼容多编译器环境，代码需要提供能够进行选择编译器的配置参数。

ECU-resource saving 资源利用率。

由于硬件资源的有限性，FC 软件设计需要根据功能需求和应用场景，考虑硬件资源利用率，在空间（FLASH 或 RAM）和时间（CPU）之间取得平衡。

6.1 ISO 26262-6 映射关系

Topics to be covered by modelling and coding guidelines

	ASIL 要求	参考本文档章节
1a	Enforcement of low complexity	6
1b	Use of language subsets	6
1c	Enforcement of strong typing	6
1d	Use of defensive implementation techniques	6
1e	Use of well-trusted design principles	6
1f	Use of unambiguous graphical representation	不适用
1g	Use of style guides	6

1h	Use of naming conventions	6
1i	Concurrency aspects	6

Design principles for software unit design and implementation

	ASIL 要求	参考本文档章节
1a	One entry and one exit point in subprograms and functions	6
1b	No dynamic objects or variables, or else online test during their creation	6
1c	Initialization of variables	6
1d	No multiple use of variable names	6
1e	Avoid global variables or else justify their usage	6
1f	Restricted use of pointers	6
1g	No implicit type conversions	6
1h	No hidden data flow or control flow	6
1i	No unconditional jumps	6
1j	No recursions	6

6.2 代码框架规范

代码框架规范规定了 FC 代码文件结构和文件中的头尾注释信息内容。

- 按照规范设计 FC 文件结构，文件内容和文件包含关系。

具体规范参考本文 FC 文件结构章节。

- 按照规范设计 FC 文件内注释头尾（版本信息）。

FC 的代码文件（所有的.c 文件和.h 文件）需要有文件注释头和注释尾（由 FC 架构工具生成）。注释头中需要正确编写“FC 版本号”和“FC 作者名”。注释尾中需要正确编写“FC 架构版本信息”（由 FC 架构工具生成）和“FC 代码版本信息”。

“FC 版本号”字段定义格式参考《G-C045 软件模块命名规范》中的 FC 版本相关章节。

“FC 作者名”字段为 FC 开发者姓名，使用英文名或拼音（由于会存在重名，推荐使用金

脉邮箱用户名)。

“FC 架构版本信息”需要简述当前 FC 架构内容和对比上一版更新差异 (非初始版本)。

“FC 代码版本信息”需要简述当前 FC 代码功能内容和对比上一版更新差异 (非初始版本)。

注意: 如果没有使用 FC 架构工具生成, 需要参考其他已使用 FC 架构工具生成的 FC 代码, 手写文件注释头和注释尾。

6.3 代码格式规范

代码格式规范规定了代码书写外观和样式 (非软件设计规范)。

- FC 代码命名规范。

所有元素 (定义, 变量, 函数等) 命名需要符合《G-C046 软件接口命名规范》。

- FC 代码块排版。

排版按照各类文件的代码模板注释块划分排序。例如 Code 文件排序为: 头文件包含→局部变量定义→内部函数声明→内部函数定义→接口函数定义。相应代码内容应写在指定的代码块后。代码模板参考附件。

- 按照规范增加详细设计解析标示符和注释。

.h 文件中的类型声明, .c 文件中的变量定义需要增加详细设计解析标示符, 并且正确编写注释。.c 文件中的全部函数, 需要正确编写注释头 (接口函数注释头由 FC 架构工具生成, 内部函数需要手写)。具体规范参考线上 Polarion→Help→FC 空间→详细设计解析工具代码注释关键字。

注意: 如果没有注释关键字和正确编写注释, 详细设计解析工具不能提取, 需要手写详细设计相应部分内容。

代码样式模板参考附件。

- 单行代码字符限制。

单行代码不超过 120 个字符, 超过限制需要拆分多行或使用\符号换行。

*注意: 代码模板注释/***.....***/宽度为 120 个字符, 用于编码时对比检查。*

超限场景和处理方式分为两种:

1: 函数定义、声明、调用超限, 将形参拆分到单行, () 和每个形参单独占一行 (类似代码块缩进写法), 不需要使用\符号。

注意: 由于函数声明比函数定义多出 extern 和一个缩进, 可能会出现函数声明超限, 但

函数定义未超限的情况，为保持统一格式，函数定义和声明都需要拆分多行处理。函数的调用也可能出现超限（代码缩进较深或实参名较长时），同样需要拆分多行处理。

2: 变量定义和运算表达式超限，一般为标定变量超限（定义时赋初值的宏字符较长），或全局变量数组超限（数组长度宏字符较长），换行原则为不要从中间拆分变量类型或变量名等有意义的字符串，运算符和赋值符号保留在当前行，后面使用\符号，后续内容写在下一行（换行），所有换行都需要基于第一行缩进一次。

注意：含有多个逻辑/关系运算符的语句，()中的高优先级运算在不限的情况下，不用换行，提高代码可读性。

代码样式模板参考附件。

- 缩进和空格的用法。

新指令模块使用缩进，例如{}里的代码需要缩进（由于文本编辑软件不同，需要确认将TAB缩进设置为4个空格）。

函数的返回值和函数名之间使用TAB缩进，多个形参之间使用空格。

#include 后使用空格再写"xxx.h"。包含的.h文件要求和实际文件后缀大小写保持一致。

运算符前后使用空格，if-else/while/for/switch-case等指令后使用空格。

代码样式模板参考附件。

- 代码块格式。

if-else/while/for/switch-case代码块必须以“{”作为起始，且“{”必须独占一行，代码块必须以“}”作为结尾，且“}”独占一行，代码块的“{”与“}”应该保持竖直方向对齐。

一个代码块（变量定义块，函数块，循环块，判断块，功能块等）后面应该空一行，不要有多余的空行。

注意：if和else需要成对使用，即使else代码为空。

注意：为了提升代码可读性，循环计数尽量使用从0开始递增，增量为1。

注意：switch判断变量的类型要和case值的类型一致。必须有default分支。

代码样式模板参考附件。

- 指针标识符*靠近数据类型。

对于指针变量, 标识符*和指向数据类型应看作一个整体, 标识符*应靠近数据类型。局部变量定义举例: `uint32* Var_pu32 = 0U;`

*注意: 此规范和 FC 架构设计中的函数输出形参 * 位置保持一致。*

代码样式模板参考附件。

- 每个变量的定义必须单独占一行, 不允许使用逗号操作符在一行定义多个变量。
- 注释规范。

注释需要使用 `/**/` (不要使用 `/**/` 嵌套)。注释位置在代码行右侧或者代码块上方。

注意: 右侧注释不会被 Code Metrics 工具 (Polyspace) 统计为有效注释行。

注释在代码上方时, 需要和对应的代码块对齐。注释在代码右侧时, 使用缩进间隔。

注释使用英文 (某些文本编辑界面会把中文字符变为乱码以至无法阅读, 甚至破坏 `/**/` 符合导致编译不通过)。

注释要正确描述, 易理解并且是必需的 (不要写和代码同样的无效注释)。

无效的注释例如:

```
Cnt_u32 = 1U; /*set Cnt_u32 value to 1*/
```

代码样式模板参考附件。

- 删除已注释掉的代码, 带有提示说明的配置示例代码除外。

6.4 代码设计规范

代码设计规范包含了禁止规则 (不允许使用)、强制规则 (必须使用) 和推荐设计方法, 部分规则和静态检查规范重合但不冲突。

代码设计要通过编译, 并且无 warning 项。

代码设计要满足静态检查规范的要求, 静态检查规范中的要求不在此文档中详述。

代码设计禁止和强制规则的目的, 包括且不限于, 减少不必要的灵活性, 保证代码简单而少出错。去除使用编译器相关语句, 增加软件可移植性。固定部分软件格式, 避免曾经出现过的 bug 产生。

注意: 所有规则在实际 FC 开发时需认真应对和考虑, 但所有规则都不是完全强制执行, 且不允许任何例外。如果实际 FC 开发过程中遇到规则与设计目的冲突(如资源使用、效率等), FC 代码需要对违反项说明合理解释(justification), 并由专家评审小组评审确定是否通过。

6.4.1 静态检查规范

FC 代码需要满足 MISRA C:2012 的 required 项和 mandatory 项要求,文档参考《G-C068-001 MISRA C 2012_检查项清单》。FC 代码需要满足 HIS 代码度量标准 (有小部分差异),文档参考《HIS Source Code Metrics》。

信息安全相关 FC 代码,除以上要求外,还需要满足 SEI CRET C 的 rule 项要求,文档参考《G-C108 安全编码规范》

以上要求在 Polyspace 中都有检查支持,文档参考《G-C068 代码静态分析操作手册》。

HIS 要求 (Polyspace 检查支持) 简述如下:

度量项	描述	规范要求
Comment Density	注释密度,指令代码和注释的比例。	≥ 40
Number of Paths	函数内部的路径数量	≤ 80
Number of Go to Statements	函数 goto 语句数量	0
Cyclomatic Complexity	函数圈复杂度	< 10
Number of Calling Functions	函数调用者的数量(调用此函数的函数,不计重复)。	≤ 5
Number of Called Functions	函数被调者的数量(函数内部调用其他函数,不计重复)。	≤ 7
Number of Function Parameters	函数形参数量	≤ 5
Number of Instructions	函数指令数量	≤ 50
Number of Call Levels	函数控制流结构的最大调用深度。	≤ 4
Number of Return Statements	函数 return 语句数量	≤ 1
Number of Recursions	函数递归调用数量(直接或间接)	0
Number of Direct Recursions		

注意: 金脉代码规范和 HIS 差异项如下,

金脉代码规范注释密度要求大于等于 40, 高于 HIS 规范。

金脉代码规范对 HIS 的 Language Scope 项不做要求。

6.4.2 禁止规则

- 禁止使用的文字和信息。

禁止在代码或注释中使用客户名称, 金脉软件平台名称, 金脉项目编号等文字和信息。

- 禁止使用连接符##。
- 禁止使用联合体。
- 禁止使用位域 (microcontroller 寄存器定义除外), 位操作使用 MASK 配合整型数据进行运算。
- 禁止 jump/goto 跳转 (单片机启动代码, 汇编代码除外)。
- 禁止使用不定参数函数, 例如 printf 函数。
- 禁止使用宏代码。

宏用于定义常数, 空指针, 不带参函数, 或编译器相关指令 (inline) 等, 禁止使用宏函数, 或宏替代的代码 (运算, 判断, 赋值等)。

- 禁止使用递归函数。
- 禁止使用编译器提供的内建函数 (库函数) 和.h 文件。

使用编译器提供的代码或类型定义可能导致跨编译器平台移植问题。

- 禁止使用动态内存分配函数。
- 禁止使用全局变量接口, 所有接口使用函数实现。
- 禁止使用隐式的数据类型转换, 不同类型的变量运算时, 使用强制类型转换实现。
- 禁止改变指针类型的操作 (获取其他类型数据地址), 如果功能有必要, 需要有代码保证读写对齐, 保证没有间隙。
- 禁止使用编译器指令直接定义变量或函数地址。

不要使用 _at_ 或类似的编译器指令直接定义变量或函数地址。单片机启动代码, 特殊需求的汇编代码除外。

- 禁止在代码中依靠编译器默认规则定义变量和函数的 memory 段, 或者使用 pragma 操作编译器 (MemMap.h 文件除外)。

所有变量和函数需要通过宏定义 memory 段名, 由 FC 的 MemMap.h 或集成的 MemMap.h 完成编译器相关指令编写。

- 禁止使用隐式的与 0 比较判断。

错误示例:

```
if (Valid_b)
while (Cnt_u8)
```

修改为使用显式的比较判断, 例如:

```
if (Valid_b != FALSE)
while (Cnt_u8 != 0U)
```

- 禁止函数内部定义 static 变量。

6.4.3 强制规则

- 编码中要求使用 AUTOSAR 标准的数据类型 (Platform_Types.h 文件中的数据类型), 不要使用编译器默认的类型

- 防止除 0 运算。

整型运算前要判断保证除数不等于 0。浮点运算防止除极小的数, 运算前要判断除数大于极小数范围 (浮点数不能使用等于判断)。

- 变量赋值的数值类型和后缀。

通常使用两种, 整型和单精度浮点型, 需要增加后缀 (大写)。

无符号整型数需要 U 后缀, 例如:

```
#define GP_DEMOFC_CHL_NUM ((uint16)10U)
```

有符号整型没有后缀, 例如:

```
#define GP_DEMOFC_TEMP ((sint16)20)
```

单精度浮点数需要 F 后缀, 单精度浮点数即使小数部分为 0 也要写出来, 例如:

```
#define GP_DEMOFC_VOL_GAIN ((float32)10.5F)
#define GP_DEMOFC_VOL_OFFSET ((float32)2.0F)
```

- 结构体定义成员位置排序。

成员位置需要按照位宽从大到小排列方式, 减少间隙, 排列顺序为:

指针→float32→枚举/uint32/sint32→uint16/sint16→uint8/sint8/boolean

注意: 当结构体 structure A 的成员为另一个结构体 structure B 时, 有以下两种实现方案供选择 (方案也适用于多个结构体包含嵌套的应用场景)。

方案 1, 使用指针, structure A 中定义成员变量为指针, 指向 structure B。

方案 2, 根据成员结构体 structure B 的首个成员变量位宽决定, 排在 structure A 同类

位宽成员最后。

- 内部全局变量定义要求。

FC 内部全局变量在定义时不用赋值（使用 .bss 类型），也不要赋初值 0。某些编译器会将赋初值为 0 的变量归类为带初始值变量（.data）。

```
static uint32 Gp_Demo_TestVar1;
```

在 FC 初始化函数中需要对内部全局变量赋初值，不要依赖编译器的库函数（例如 _c_init）对变量（使用 .data 类型）赋值来完成内部全局变量初始化。

const 变量和 const 数组定义时必须填入初始值（数组所有元素都要填入初始值）。

- 局部变量在使用前必须赋初值。

FC 函数内的局部变量在使用前必须赋初值，其中指针变量赋值为同类型的已有变量地址，后续操作时可以更改为需要读写的同类型变量地址。推荐在局部变量定义时赋初值。

- 局部变量定义的 size 限制。

FC 函数内的单个局部变量的 size 如果超过 50 字节，或者全部局部变量总 size 超过 200 字节时，需要将此单个变量或者较大 size 的变量定义为内部全局变量，以节省函数消耗堆栈空间。

- for 循环或 while 循环的计数值变量必须为 uint32 类型。

使用 for 循环或 while 循环语句时（计数判断），计数值变量必须为 uint32 类型（注意代码中计数阈值需要强转后再比较）。避免出现计数值变量类型的上限比阈值小，导致出现死循环情况。

```
uint32 Cnt_u32 = 0U;
```

```
for (Cnt_u32 = 0U; Cnt_u32 < (uint32)GP_IOMCUADC_GRP_NUM; Cnt_u32++)
```

- 指针的限制使用方式。

使用指针寻址时，限制指针本身不要动（值不变），通过指针加偏移量寻址，必要时对偏移量做范围限制或检查来保证寻址不会越界。

- 使用括号显式的定义运算优先级。

当运算含有多个运算符时，使用括号显式的定义运算优先级，不依靠运算符默认优先级。

- 定义数组或结构体数组时，数组长度使用有相关含义的宏定义。

宏定义可以用于 for 循环阈值和范围检查。读写数组元素时，需要对索引值做限定操作，

防止数组越界，导致非预期篡改其他变量。

当使用循环体读写数组元素，计数值作为数值索引值，数组长度宏作为计数阈值时，这种情况下不需要额外代码对索引值做限定。

当读写数组元素的索引值是经过计算后的变量时，需要判断变量是否在数组长度范围内。

- 多个变量运算防止溢出。

可能存在中间值溢出的多个变量运算，需要将运算分拆到多行代码，且使用更大位宽的临时变量来保存计算中间值。

例如：

```
A_u8 = (B_u8 * C_u8) / D_u8;
```

需要拆分两步运算，并定义更大位宽的临时变量 `Tmp_u16` 做缓存，注意需要使用强制类型转换。

```
Tmp_u16 = (uint16)B_u8 * (uint16)C_u8;  
A_u8 = (uint8)(Tmp_u16 / (uint16)D_u8);
```

注意：更大位宽临时变量的加入，便于调试观察，同时避免低位宽 MCU (8bit/16bit MCU) 运算时（加法或乘法运算），中间值自动丢弃了超过 CPU 数据寄存器宽度的部分。

- 内联函数的使用。

FC 内部多次被调用且运行时间较短的代码（函数本身指令周期比调用函数花费的指令周期少）需要使用内联函数。

使用内联函数时，需要使用 FC 模板文件定义的 `inline` 宏，适配不同编译环境，增加移植性。`inline` 函数需要写在当前.c 文件或新增.h 文件中（可新增独立的 `inline` 头文件）。

- 函数只能有一个出口。

- 接口函数需要做防御性编程。

FC 接口函数，需要对输入形参（输入 ID 类，输出指针类等）做有效检查，形参检查结果决定返回值为 `E_OK` 或 `E_NOT_OK` (`Std_ReturnType` 类型)，返回值赋初值为 `E_NOT_OK`。形参检查为无效时，函数不执行功能，退出并返回 `E_NOT_OK`。

FC 内部函数返回值不要使用 `Std_ReturnType` 类型，FC 内部函数不需要防御性编程。

注意：防御性编程主要针对集成时外部输入的错误形参，和错误的调用时序，错误将返回给外部调用者。FC 内部函数的调用时序正确性和形参输入有效性由 FC 开发和 FC 测试保证。

- 使用临界区保护变量。

当 FC 接口函数接收到外部输入数据，保存到 FC 内部全局变量后再计算时。代码需要按以下顺序设计：

进入临界区。

将 FC 内部全局变量赋值到函数局部变量。

退出临界区。

函数局部变量参与运算（保证运算过程不被接口函数重入影响），运算完成后。

进入临界区。

将函数局部变量赋值到 FC 内部全局变量。

退出临界区。

另外，对 FC 内部全局变量赋值，如果多个变量有数据一致性要求的（要求是存储同一时刻的值），可增加临界区保护。

注意：临界区为配置宏，当集成调用明确没有重入风险时，配置宏为空（提升代码速度）。临界区配置代码一般只作用于当前核，代码为挂起/恢复 `interrupt`。当 FC 接口被多核调用并且 FC 不支持多核可重入时，临界区代码一般为等待获取/释放 `spinlock`。

6.4.4 推荐设计方法

- 提高多个分支程序的效率。

代码有多个分支时，根据 FC 功能分析，将最大概率判断为真的分支条件放到判断语句（if-else/switch-case）的首次判断，减少判断跳转。同样，后续分支条件可以按概率从大到小排序。

例如，正常运行功能的分支条件靠前，判断故障并处理的分支条件靠后，常用的配置代码分支条件靠前，特殊处理代码分支条件靠后。

- 形参和局部变量的个数。

FC 函数代码需要注意，FC 函数的形参和局部变量尽量不要超过 4 个（`tricore` 性能）。

此时形参和局部变量将使用 CPU 寄存器处理，而不是创建在堆栈上，减少堆栈开销并提升运行速度。

- 避免在函数内部使用大量的全局变量运算。

由于 `far` 寻址方式定义的变量运算的汇编代码较多，对于同一个全局变量的多次运算（3

次及以上), 推荐两种方式:

1: 在函数开始后, 将全局变量值写入局部变量, 由局部变量参与运算, 在函数结束前, 将局部变量值写回全局变量

2: 在函数开始后, 定义指针局部变量指向此全局变量, 由指针参与运算。

基于方式 2 的延伸, 推荐 FC 内部全局变量定义使用结构体, 而非零散的单个变量, 方便使用结构体指针获取到所有变量。

注意: 此方法特别适用于支持多核运行 FC 的全局变量定义。

指针赋值和全局变量赋值两种方式代码效率差别如下图, 绿框中代码为指针寻址赋值, 红框中代码为全局变量赋值。

```
#if (GP_IOMCUADC_DEV_ERROR_DETECT == STD_ON)
    Gp_IoMcuAdc_RunTimeType *Rt_ptst = &Gp_IoMcuAdc_rtContainer_1tst;
    movh.a    a15,#0x7000
    lea       a15,[a15]0x44    ; Rt_ptst,[a15]68
    boolean ErrFlag_b = FALSE;
#endif

#if (GP_IOMCUADC_SPEC_MCUIF == GP_IOMCUADC_SPEC_MCUIF_IFX2GMCAL)
    volatile    uint32 ValidFlag_u32 = 0U;
    movl6      d15,#0x0
    stl6.w     [a10],d15
#endif

/*check init status and argument validity*/
#if (GP_IOMCUADC_DEV_ERROR_DETECT == STD_ON)
    if (Rt_ptst->InitStu_te == Gp_IoMcuAdc_InitStuInitied_e)
        ld16.w  d15,[a15]
        jne16   d15,#0x1,0x800E376C
    {
        if ((Id_u16 < GP_IOMCUADC_CHL_NUM) &&
            jge.u  d4,#0x7,0x800E3758    ; Id_u16,#7,0x800E3758
            (AdcRes_pu16 != GP_IOMCUADC_NULL_PTR) &&
            jz16.a  a4,0x800E3758    ; AdcRes_pu16,0x800E3758
            (AdcResStu_pb != GP_IOMCUADC_NULL_PTR))
            jz16.a  a5,0x800E3758    ; AdcResStu_pb,0x800E3758
        {
            ErrFlag_b = FALSE;
            movl6   d0,#0x0
            jl16    0x800E376A
        }
        ErrFlag_b = TRUE;
        movl6      d0,#0x1
        Rt_ptst->ErrStu_u8 = GP_IOMCUADC_INVALID_ARG;
        movl6      d15,#0x2
        stl6.b     [a15]0x4,d15
        Gp_IoMcuAdc_rtContainer_1tst.ErrStu_u8 = GP_IOMCUADC_UNINITED;
        movh.a     a15,#0x7000    ; Rt_ptst,#28672
        lea        a15,[a15]0x44
        movl6      d15,#0x1
        stl6.b     [a15]0x4,d15
    }
    else
        il16      0x800E3774
    endif
#endif
```

- 简化判断语句的条件表达式。

if 等语句的条件表达式不要直接使用复杂算式, 需要提前计算好并存放在局部变量中, if 语句只对局部变量值做判断 (TRUE/FASLE)。

注意: 可以将判断代码放到一个内部函数实现, 函数返回 TRUE/FALSE 供调用时 if 语句

判断, 此方法会简化动态测试输入打桩。

- 代码需要去除无谓的计算部分。

例如: 固定的转换公式, 寄存器值移位和拼接等操作。可在 Excel 或其他辅助工具中提前计算好结果或者提前定义好宏, 再放到代码中, 而不是使用代码实现。

- 增加定义 FC 内部全局变量作为 FC 接口交互的实体。

定义输入变量 (内部全局变量), 在 FC 接口函数开始后, 将输入值存入输入变量中, 后续运算从 FC 输入变量取值。

定义输出变量 (内部全局变量), 在 FC 接口函数结束前, 将运算结果存入输出变量中, 输出变量作为 FC 输出值供调用者使用。

输入变量和输出变量可方便调试观察使用。

另外, 可以为每个输入变量和输出变量增加同类型的测试标定变量, 和 bypass 功能标定变量。当 bypass 功能开启时 (bypass 功能默认关闭), 测试标定变量将取代函数的输入和输出。此方法适用于集成排查和自动化测试。

7 支持/相关性文件

序号	文档名称	文档编号
1	软件接口命名规范	G-C046
2	MISRA C 2012_检查项清单	G-C068-001
3	HIS Source Code Metrics	/
4	安全编码规范	G-C108
5	代码静态分析操作手册	G-C068
6	IS026262-2018	/
7	IS021434-2021	/

8 附件



FC代码样式模板.rar



HIS-sc-metriken.1.3.1_e.rar

拟制: 何翔宇

审核: 王鹏伟

批准: 肖洋